

自然语言处理 (2025 春季学期)

三、神经网络基础



大纲

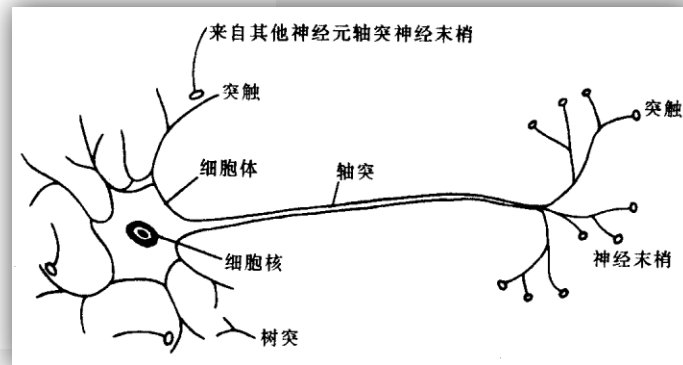
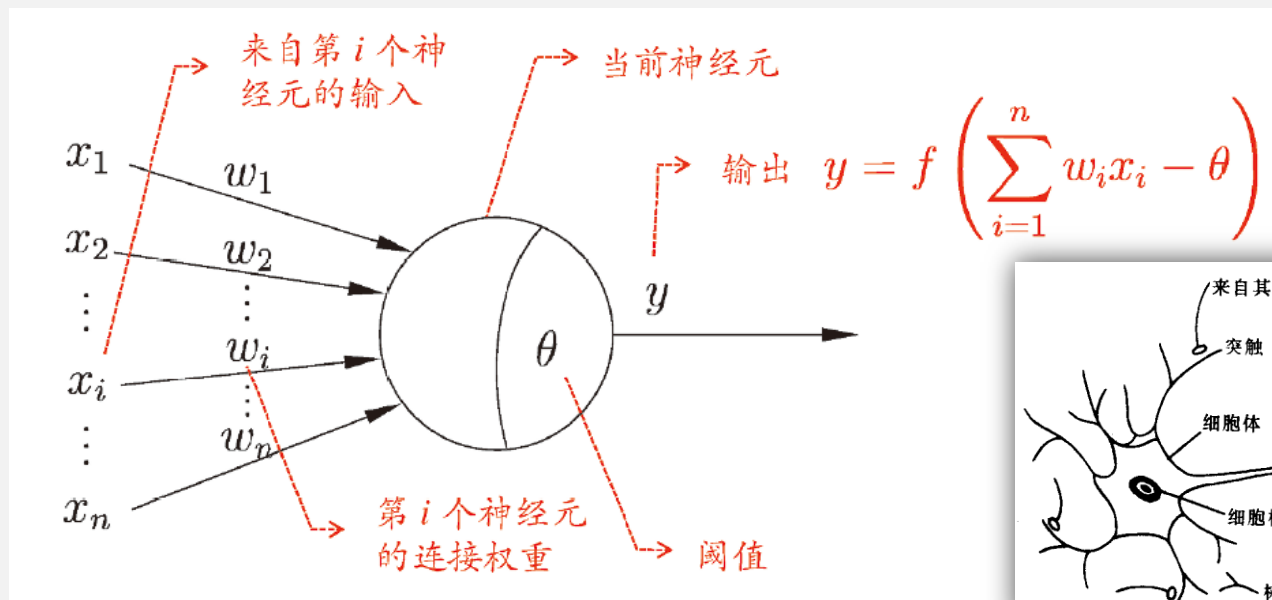
- 多层感知机
- 卷积神经网络
- 循环神经网络
- Transformer
- 神经网络的训练





“简单单元”：神经元模型

M-P 神经元模型 [McCulloch and Pitts, 1943]



神经网络学得的知识蕴含在连接权与阈值中



感知器模型

□ 感知器 (Perceptron)

- 最简单也是最早出现的机器学习模型
- 灵感直接来源于生产生活的实践

□ 面试评分

- $s = x_1 + x_2 + x_3 + x_4$
 - 如果 $s \geq t$ 则录用, 否则不录用
- 考虑面试官的经验 (权重)
 - $s = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4$

□ 感知器模型

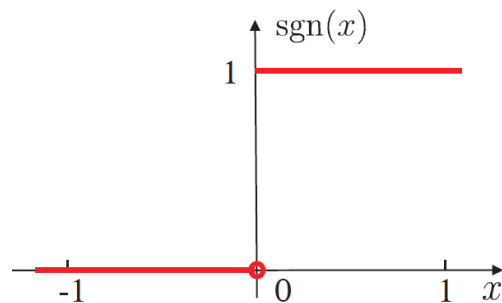
$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b \geq 0 \\ 0 & \text{else} \end{cases}, \text{ 其中 } b = -t$$





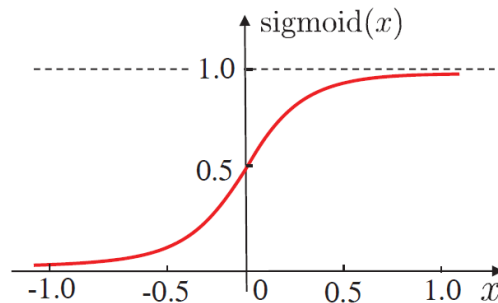
神经元的“激活函数”

- 理想激活函数是阶跃函数, **0**表示抑制神经元而**1**表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质, 常用的是 Sigmoid 函数



$$\text{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

(b) Sigmoid 函数

图 5.2 典型的神经元激活函数



Logistic回归模型

□ 线性回归 (Linear Regression)

- $y = \sum_i w_i x_i + b$

□ Logistic回归 (Logistic Regression)

- Logistic函数

$$y = \frac{L}{1 + e^{-k(z-z_0)}}$$

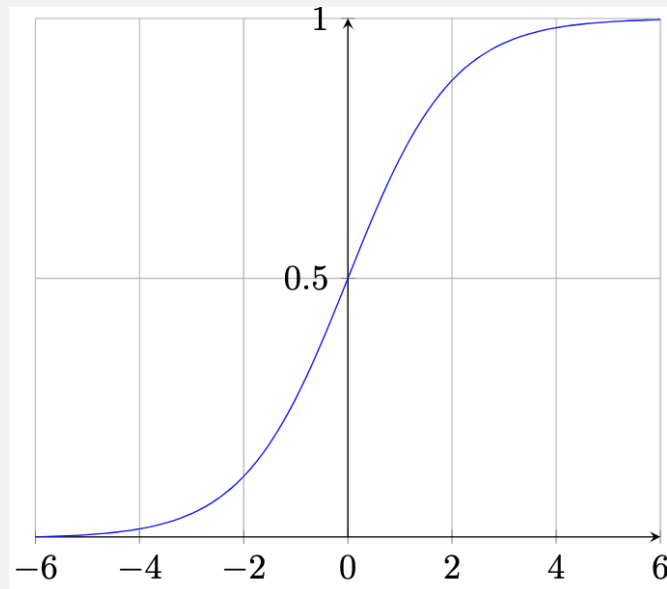
- 设 $z = \sum_i w_i x_i + b$

- 标准Logistic 函数 (Sigmoid型函数)

$$y = \frac{1}{1 + e^{-z}}$$

- 处理二元分类问题, y 为输出的概率

- 垃圾邮件过滤、褒贬识别



Sigmoid函数示意图



Softmax回归

□ 使用Softmax回归处理多元分类

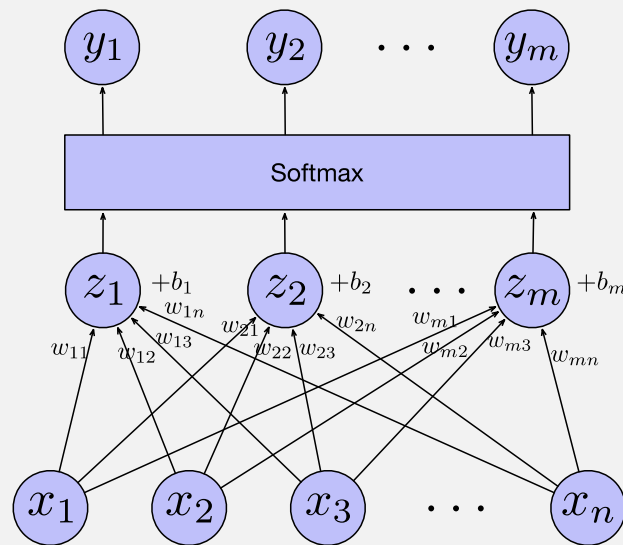
□ Softmax函数

$$y_i = \text{Softmax}(z)_i = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_m}}$$

— 其中 y_i 为第 i 个类别的概率, $z_i = \sum_j w_{ij}x_k + b$,
 w_{ij} 为第 i 个类别所对应的第 j 个输入的权重

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \left(\begin{bmatrix} w_{11}x_1 + w_{12}x_2 + \dots + w_{1n}x_n + b_1 \\ w_{21}x_1 + w_{22}x_2 + \dots + w_{2n}x_n + b_2 \\ \vdots \\ w_{m1}x_1 + w_{m2}x_2 + \dots + w_{mn}x_n + b_m \end{bmatrix} \right)$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \text{Softmax} \left(\begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \right)$$



$$\mathbf{y} = \text{Softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

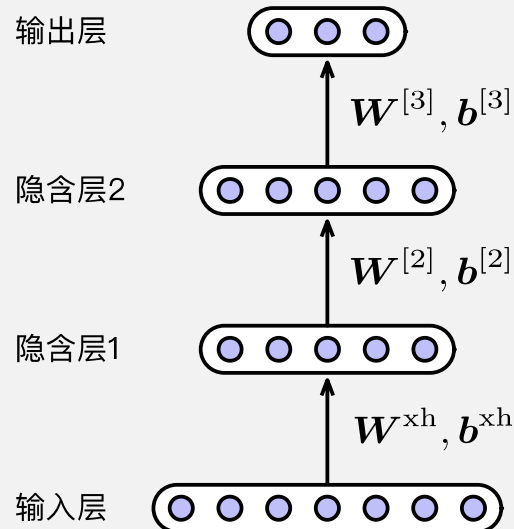
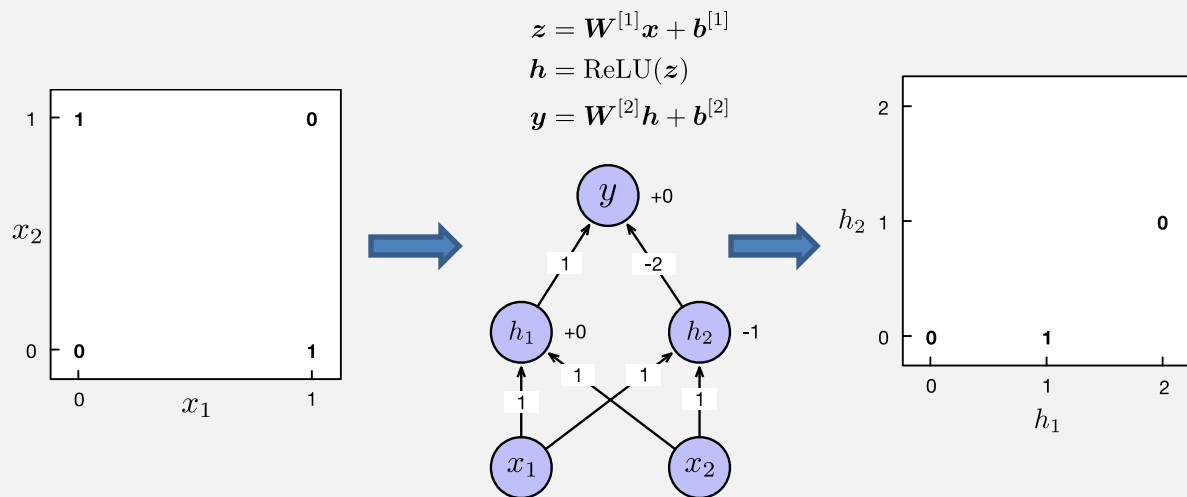


多层感知机

□ 多层感知器 (Multi-layer Perceptron, MLP)

- 堆叠多层感知器 (线性回归+非线性激活函数)
- 可解决线性不可分问题

□ 使用 MLP 解决异或 (XOR) 问题





多层感知器PyTorch实现

□ 引入PyTorch模块

— `>>> from torch import nn`

□ 线性层

```
>>> linear = nn.Linear(32, 2) # 输入32维, 输出2维
>>> inputs = torch.rand(3, 32) # 创建一个形状为(3, 32)的随机张量, 3为批次大小
>>> outputs = linear(inputs) # 输出张量形状为(3, 2)
>>> print(outputs)
tensor([[ 0.5387, -0.4537],
        [ 0.2181, -0.3745],
        [ 0.3704, -0.8121]], grad_fn=<AddmmBackward>)
```

□ 激活函数

— `>>> from torch.nn import functional as F`

```
>>> activation = F.sigmoid(outputs)
>>> print(activation)
tensor([[0.6315, 0.3885],
        [0.5543, 0.4075],
        [0.5916, 0.3074]], grad_fn=<SigmoidBackward>)
>>> activation = F.softmax(outputs, dim=1)
# 沿着第2维进行Softmax运算, 即对每批次中的各样例分别进行Softmax运算
>>> print(activation)
tensor([[0.7296, 0.2704],
        [0.6440, 0.3560],
        [0.7654, 0.2346]], grad_fn=<SoftmaxBackward>)
>>> activation = F.relu(outputs)
>>> print(activation)
tensor([[0.5387, 0.0000],
        [0.2181, 0.0000],
        [0.3704, 0.0000]], grad_fn=<ReluBackward0>)
```



多层感知器PyTorch实现

□ 自定义神经网络实现

```
class MLP(nn.Module):  
    def __init__(self, input_dim, hidden_dim, num_class):  
        super(MLP, self).__init__()  
        # 线性变换: 输入层->隐含层  
        self.linear1 = nn.Linear(input_dim, hidden_dim)  
        # 使用ReLU激活函数  
        self.activate = F.relu  
        # 线性变换: 隐含层->输出层  
        self.linear2 = nn.Linear(hidden_dim, num_class)  
  
    def forward(self, inputs):  
        hidden = self.linear1(inputs)  
        activation = self.activate(hidden)  
        outputs = self.linear2(activation)  
        probs = F.softmax(outputs, dim=1) # 获得每个输入属于某一类别的概率  
        return probs
```

```
mlp = MLP(input_dim=4, hidden_dim=5, num_class=2)  
inputs = torch.rand(3, 4) # 输入形状为(3, 4)的张量, 其中3表示有3个输入, 4表示每个  
                           输入的维度  
probs = mlp(inputs) # 自动调用forward函数  
print(probs) # 输出3个输入对应输出的概率
```





常用激活函数

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$





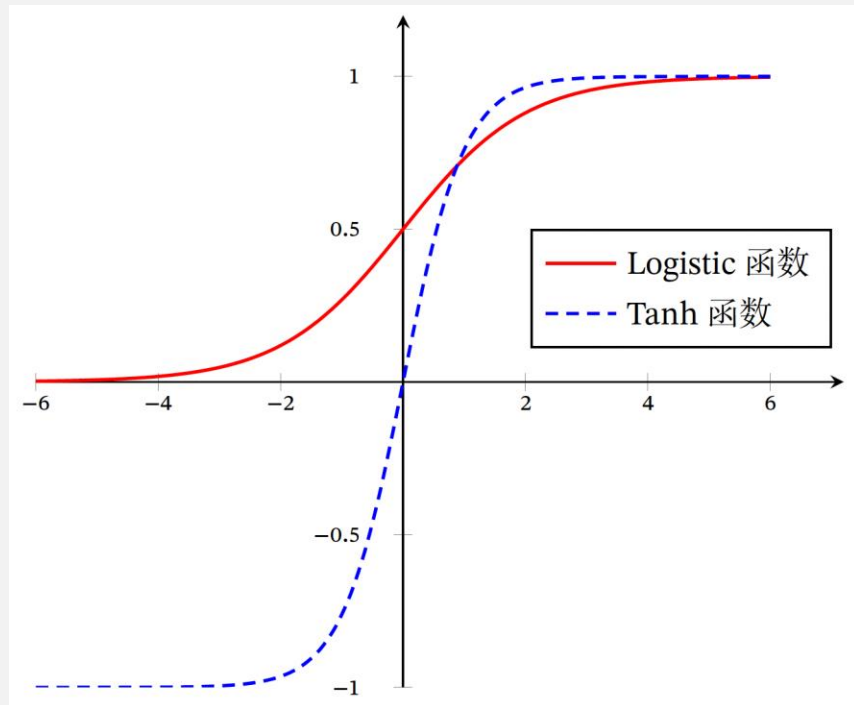
Tanh函数

- Tanh 函数也是一种 Sigmoid 型函数. 其定义为:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

- Tanh 函数可以看作放大并平移的 Logistic 函数, 其值域是 $(-1, 1)$.

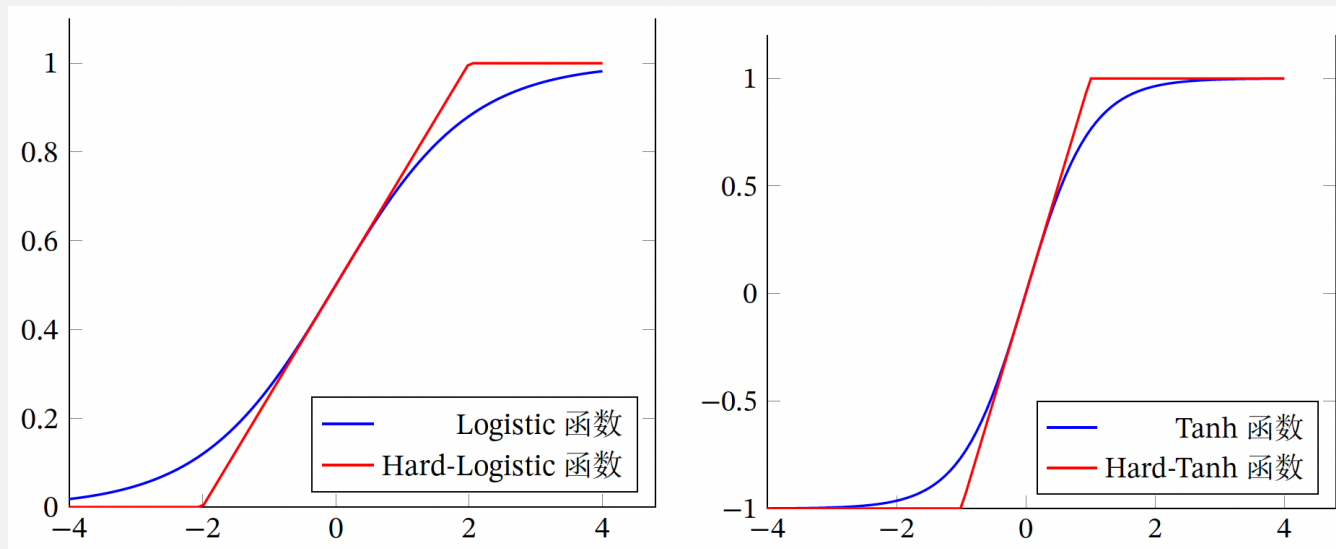
$$\tanh(x) = 2\sigma(2x) - 1$$





Hard-Tanh函数

- Logistic 函数和 Tanh 函数都是 Sigmoid 型函数，具有饱和性，但是**计算开销较大**。因为这两个函数都是在中间（0 附近）近似线性，两端饱和。因此，这两个函数可以通过**分段函数**来近似。





ReLU函数

- ReLU (Rectified Linear Unit, 修正线性单元) [Nair et al., 2010], 也叫 Rectifier 函数[Glorot et al., 2011], 是目前深度神经网络中经常使用的激活函数. ReLU 实际上是一个斜坡 (ramp) 函数, 定义为:

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$

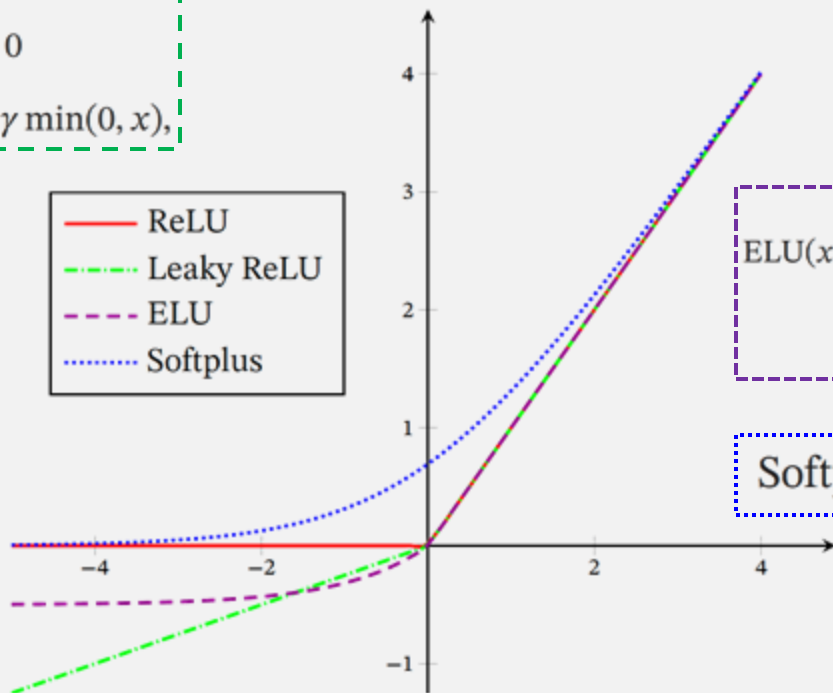
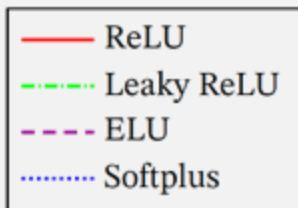
- **优点:** 采用ReLU 的神经元只需要进行加、乘和比较的操作, 计算上更加高效; 宽兴奋边界在一定程度上缓解了神经网络的梯度消失问题, 加速梯度下降的收敛速度.
- **缺点:** ReLU 神经元在训练时比较容易“死亡”, 如果参数在一次不恰当的更新后, 某个ReLU 神经元在所有的训练数据上都不能被激活, 那么这个神经元自身参数的梯度永远都会是0, 在以后的训练过程中永远不能被激活.



Leaky ReLU函数

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x),$$

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$



$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1)),$$

$$\text{Softplus}(x) = \log(1 + \exp(x)).$$





大纲

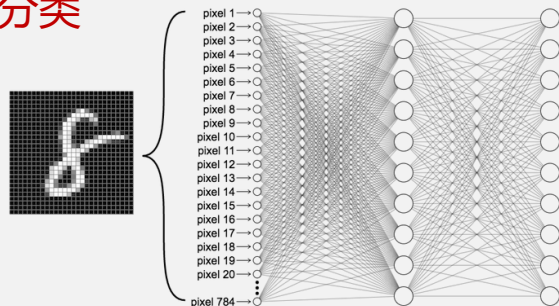
- 多层感知机
- 卷积神经网络
- 循环神经网络
- Transformer
- 神经网络的训练



卷积神经网络

□ 全连接的多层感知器无法处理输入的偏移情况

— 图像分类



— 情感分类

- 我喜欢自然语言处理。 vs. 我非常喜欢自然语言处理。

□ 解决方案

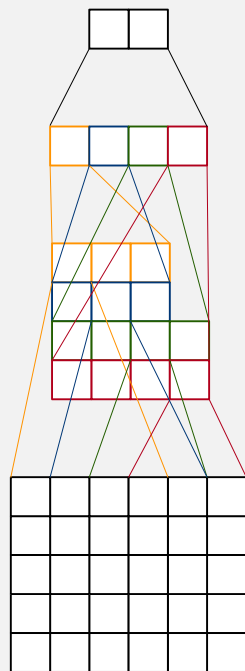
- 使用小的全连接层抽取局部特征（又称卷积核或滤波器）
 - 如遍历文本中的N-gram等
- 使用多个卷积核提取不同种类的特征
- 使用池化层将特征进行聚合（最大、平均、求和等）

全连接层

池化层

卷积层

输入层



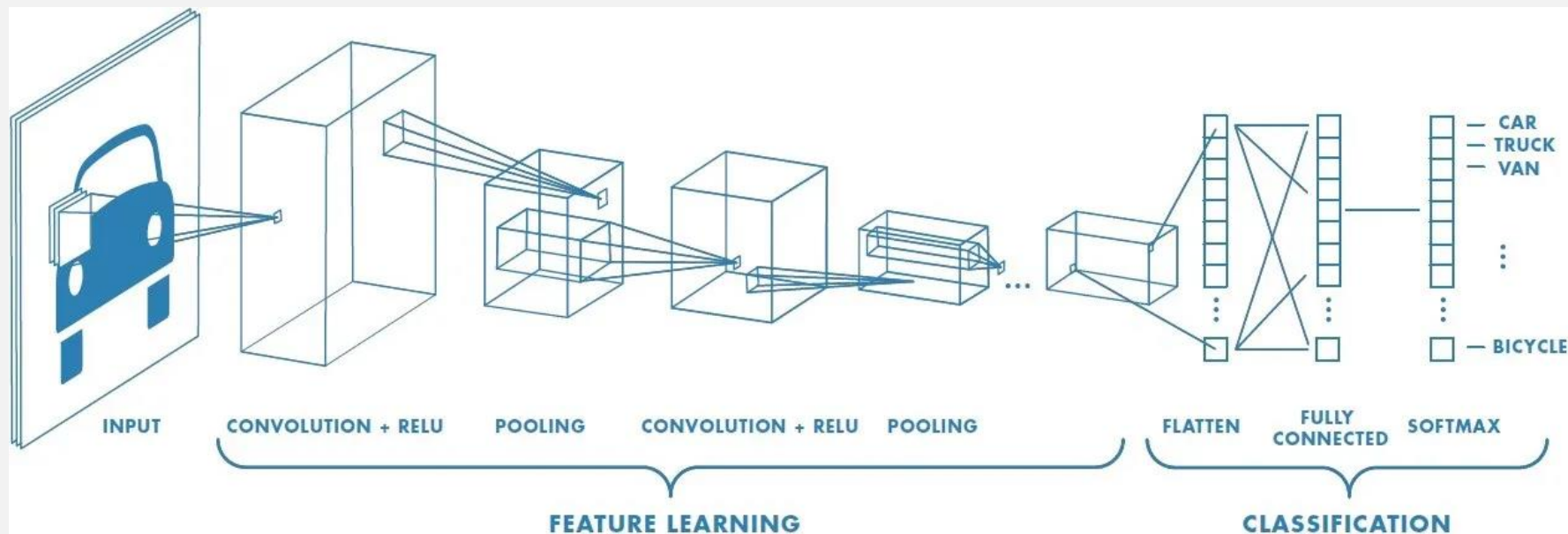
我 喜 自 语 处 。
欢 然 言 理



南京大学
NANJING UNIVERSITY

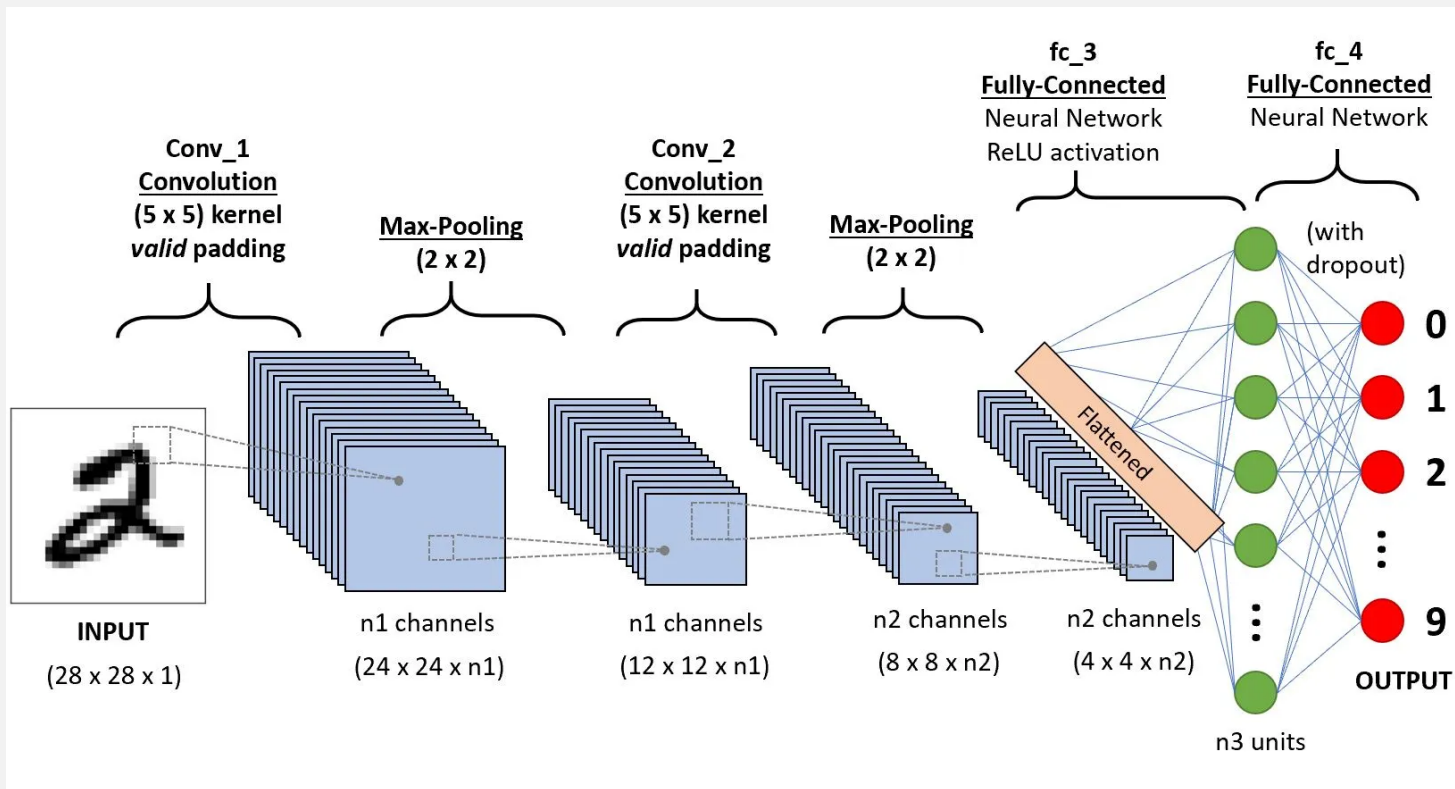


CNN 卷积神经网络





CNN 卷积神经网络





卷积层-2D

liq.opengenus.org

1	0	1	0	1	0
0	1	1	0	1	1
1	0	1	0	1	0
1	0	1	1	1	0
0	1	1	0	1	1
1	0	1	0	1	0

Input

1	0	1
0	1	1
1	0	1

Image patch
(Local receptive field)

1	2	3
4	5	6
7	8	9

Kernel
(filter)

*

31			

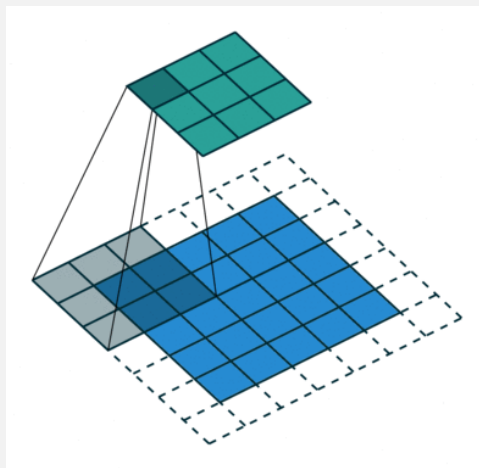
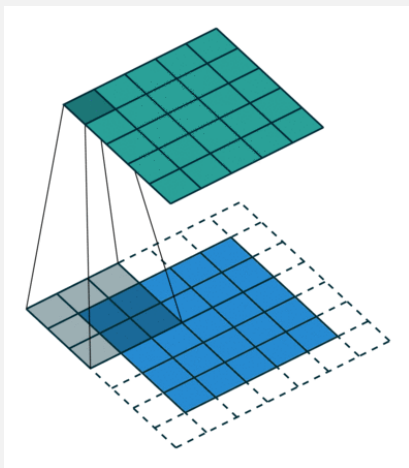
Output

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

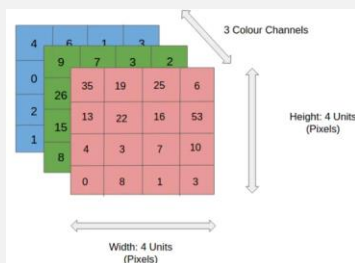
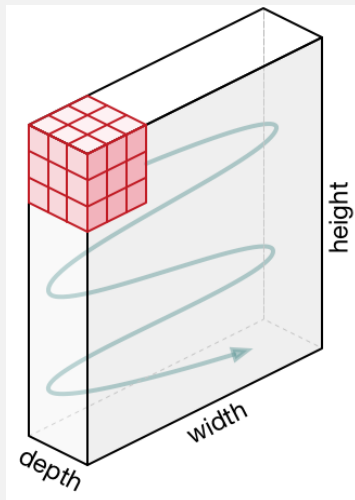
4		

Convolved
Feature





卷积层-3D



0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

308

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

-498

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

164

+

+

+ 1 = -25

Bias = 1

Output

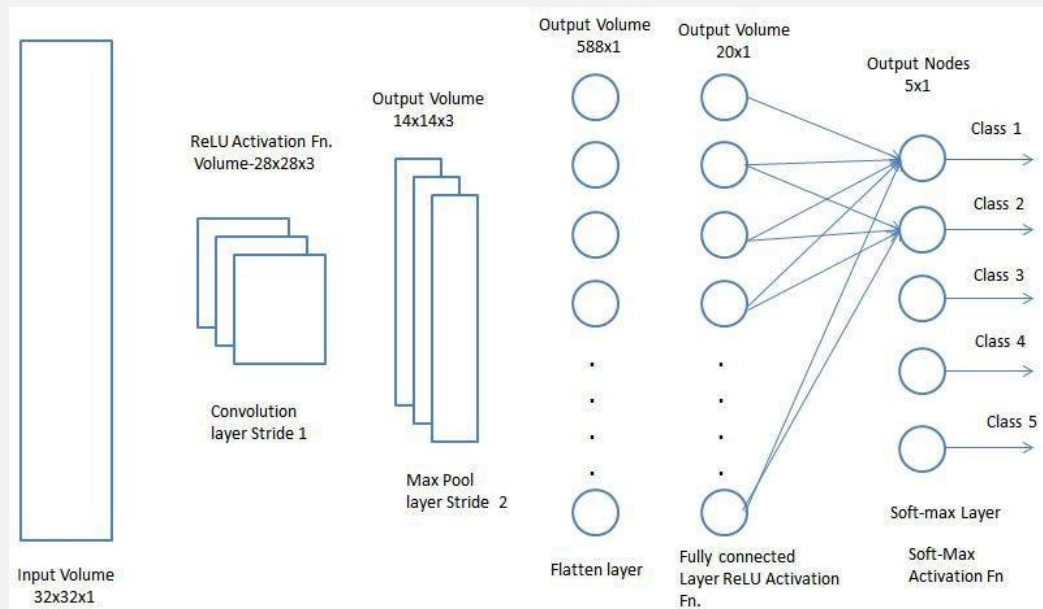
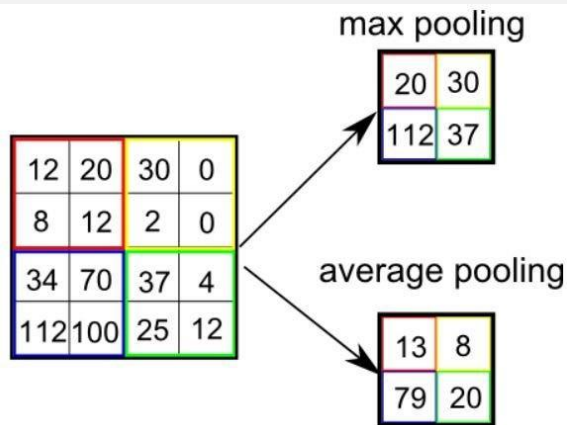
-25			...
			...
			...
			...
			...
...	...	...	...



汇聚层 & 全连接层

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

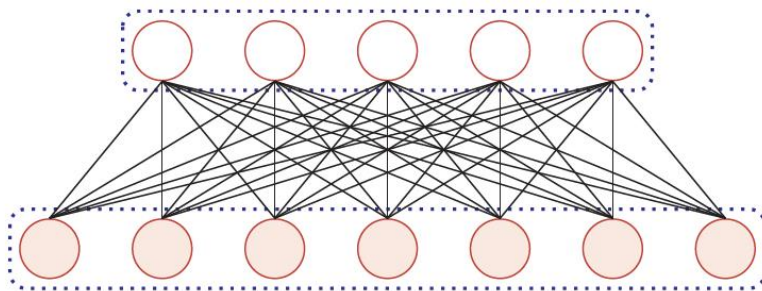
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1



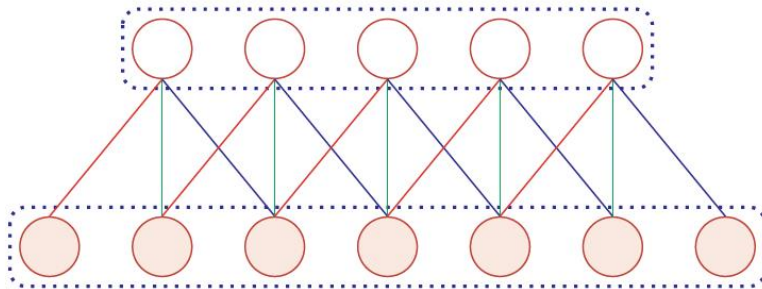


卷积层 vs. 全连接层

- 局部连接
- 权重共享



(a) 全连接层



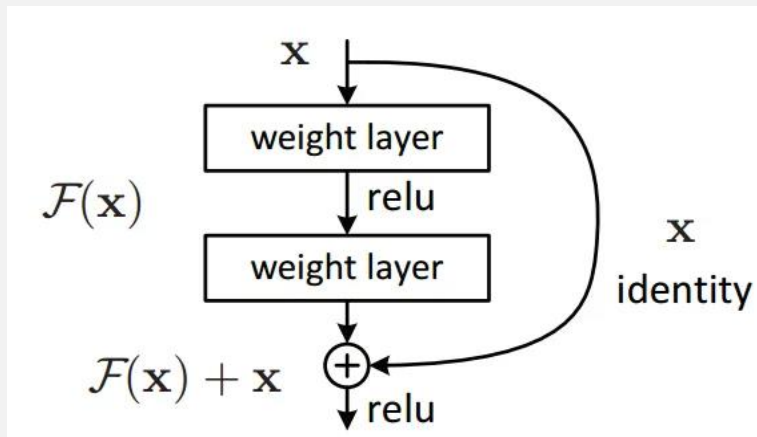
(b) 卷积层



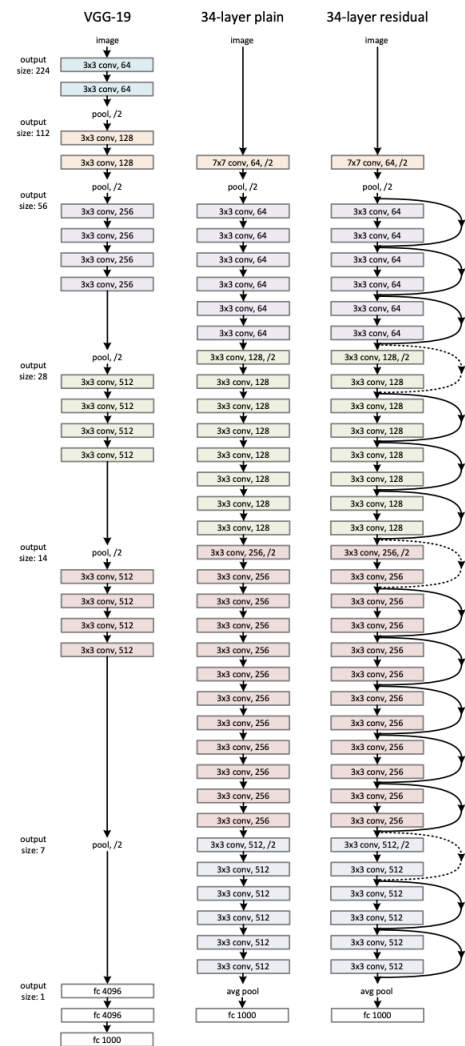


基于CNN的模型

- LeNet
- AlexNet
- VGGNet
- GoogLeNet
- ResNet
- UNet
-



$$h(x) = \underbrace{x}_{\text{恒等函数}} + \underbrace{(h(x) - x)}_{\text{残差函数}}.$$





大纲

- 多层感知机
- 卷积神经网络
- 循环神经网络
- Transformer
- 神经网络的训练



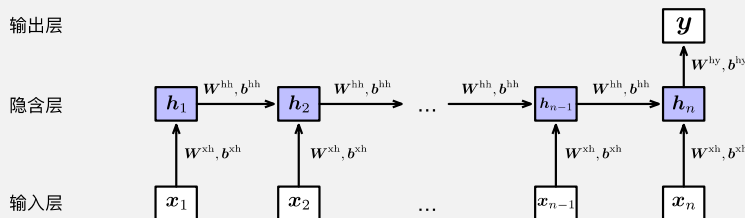


循环神经网络

❑ 卷积神经网络无法处理长距离的依赖

❑ 循环神经网络的解决方案

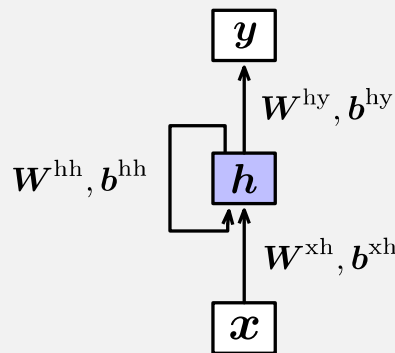
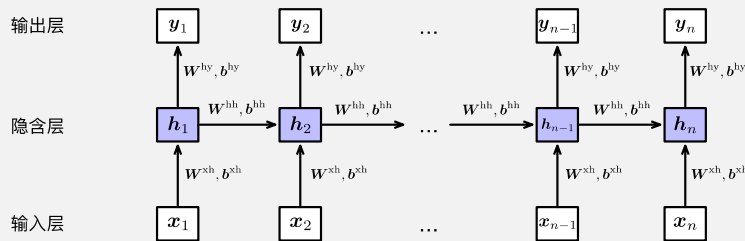
- 每个时刻的隐状态依赖于**当前时刻的输入**以及**上一时刻的隐状态**



$$h_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$

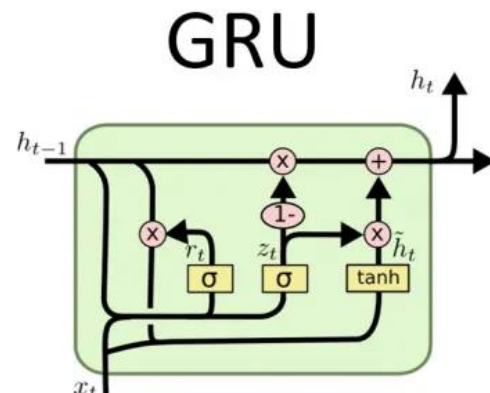
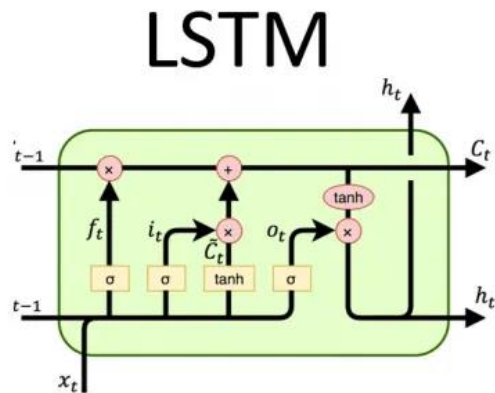
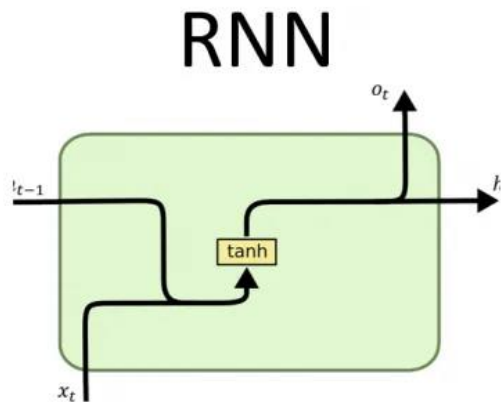
$$y = \text{Softmax}(W^{hy}h_n + b^{hy})$$

- 每个时刻的**参数共享**（“循环”的由来）
- 每个时刻也可以有**相应的输出**（序列标注）





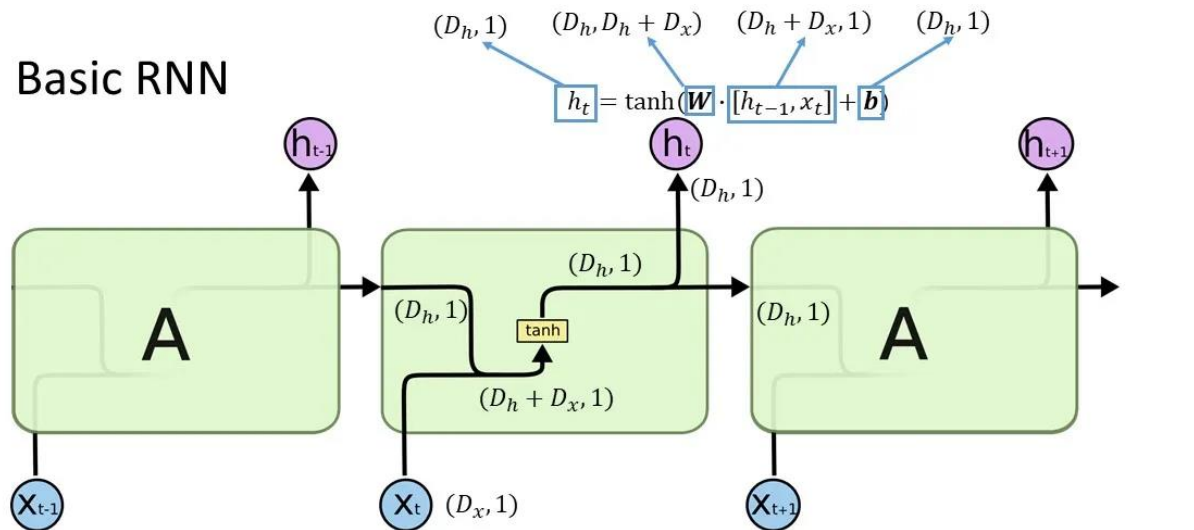
RNN 循环神经网络



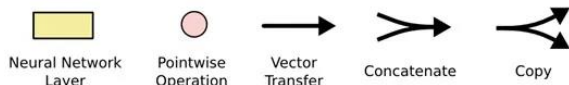


RNN 循环神经网络

Basic RNN



RNN Unit A contains:
 D_h neurons
 $D_h * (D_h + D_x) + D_h$ parameters

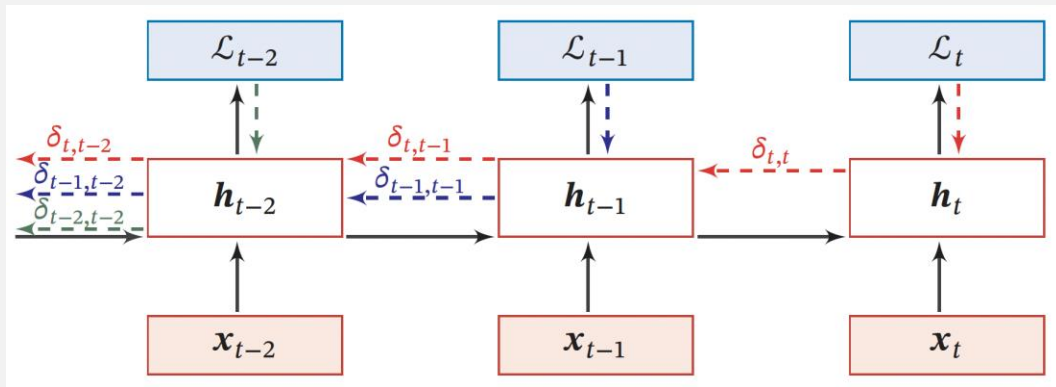




长短时记忆网络 (LSTM)

□ 序列过长时，原始的循环神经网络容易导致信息损失

— 梯度爆炸或梯度消散



$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(z_\tau)) U^\top \right) \delta_{t,t}.$$



$$\delta_{t,k} \cong \gamma^{t-k} \delta_{t,t}.$$

若 $\gamma > 1$, 当 $t-k \rightarrow \infty$ 时, $\gamma^{t-k} \rightarrow \infty$. 当间隔 $t-k$ 比较大时, 梯度也变得很大, 会造成系统不稳定, 称为**梯度爆炸问题** (Gradient Exploding Problem).

相反, 若 $\gamma < 1$, 当 $t-k \rightarrow \infty$ 时, $\gamma^{t-k} \rightarrow 0$. 当间隔 $t-k$ 比较大时, 梯度也变得非常小, 会出现和深层前馈神经网络类似的**梯度消失问题** (Vanishing Gradient Problem).



长短时记忆网络 (LSTM)

□ 序列过长时，原始的循环神经网络容易导致信息损失

- 梯度爆炸或梯度消散

□ 新的加性隐状态更新方式

$$u_t = \tanh(W^{xh}x_t + b^{xh} + W^{hh}h_{t-1} + b^{hh})$$

$$h_t = h_{t-1} + u_t$$

long term

short term

- 相当于直接将 h_k 与 h_t ($k < t$) 进行了跨层连接

$$\bullet \quad h_t = h_{t-1} + u_t = h_{t-2} + u_{t-1} + u_t = h_k + u_{k+1} + u_{k+2} + \dots + u_t$$

□ 进一步改进

$$f_t = \sigma(W^{f,xh}x_t + b^{f,xh} + W^{f,hh}h_{t-1} + b^{f,hh})$$

$$h_t = f_t \odot h_{t-1} + (1 - f_t) \odot u_t$$

- 遗忘门：考虑旧状态 h_{t-1} 和新状态 u_t 的贡献

$$i_t = \sigma(W^{i,xh}x_t + b^{i,xh} + W^{i,hh}h_{t-1} + b^{i,hh})$$

$$h_t = f_t \odot h_{t-1} + i_t \odot u_t$$

- 输入门：独立控制 h_{t-1} 和 u_t 的贡献

$$o_t = \sigma(W^{o,xh}x_t + b^{o,xh} + W^{o,hh}h_{t-1} + b^{o,hh})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot u_t$$

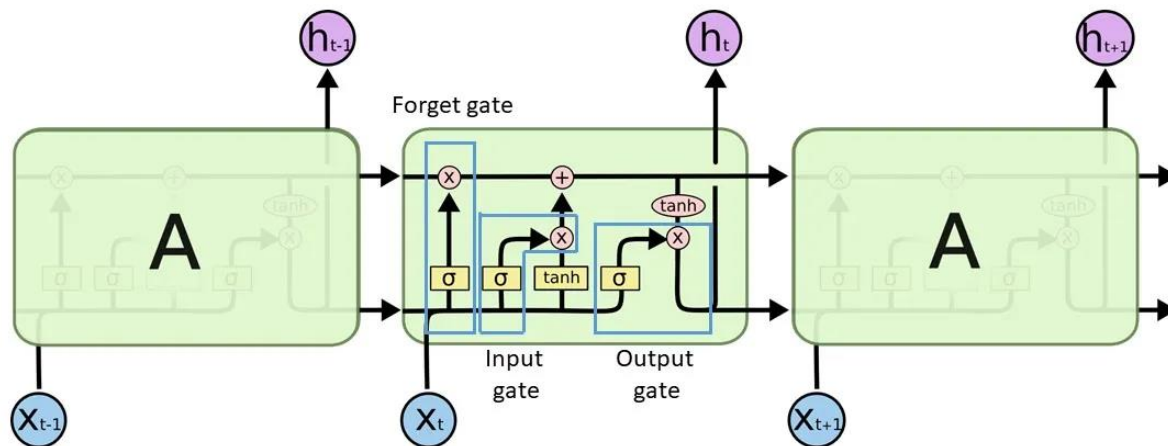
- 输出门：对输出进行控制

$$h_t = o_t \odot \tanh(c_t)$$



LSTM

LSTM



RNN Gate contains (in LSTM or GRU):

- A sigmoid neural network layer
- A multiplication operator

Neural Network
Layer

Pointwise
Operation

Vector
Transfer

Concatenate

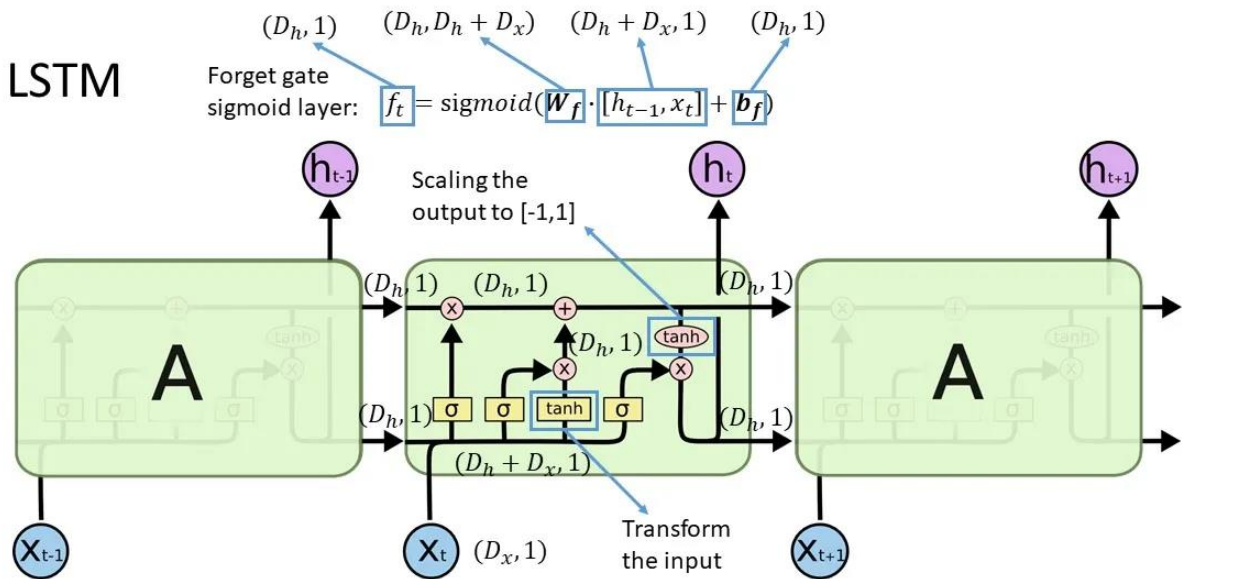
Copy



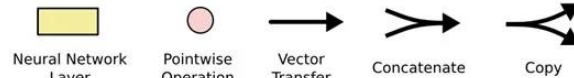


LSTM

LSTM

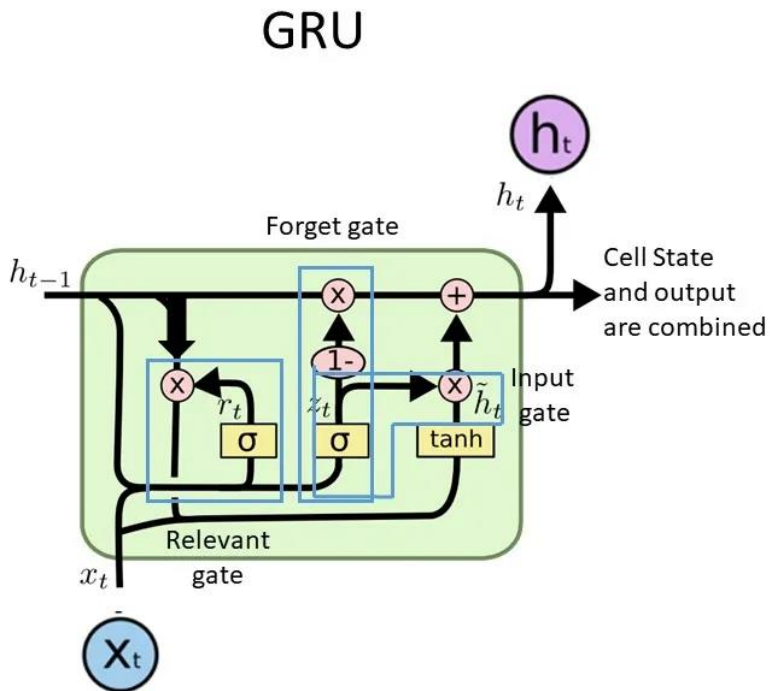
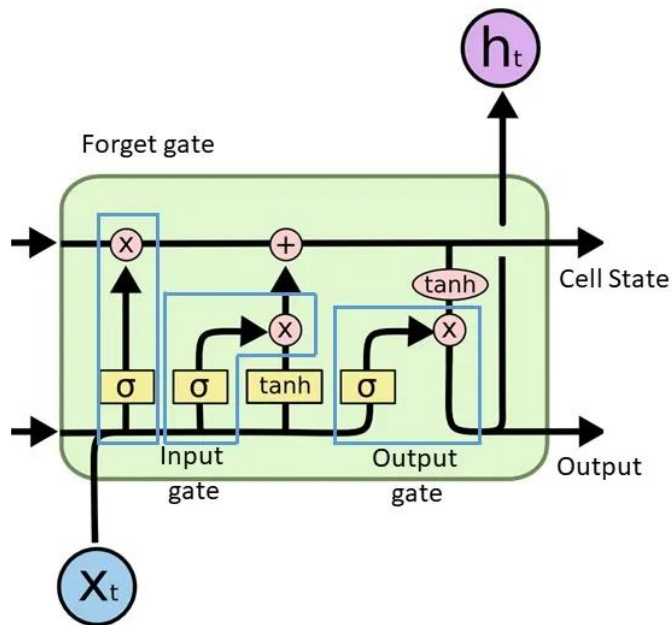


A LSTM Unit A contains:
 $4 * D_h$ neurons
 $4 * D_h * (D_h + D_x) + 4 * D_h$ parameters





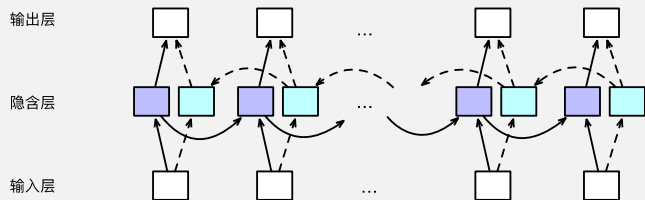
GRU



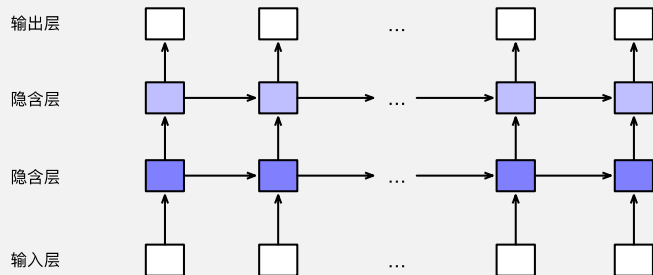


循环神经网络的应用

□ 双向循环神经网络

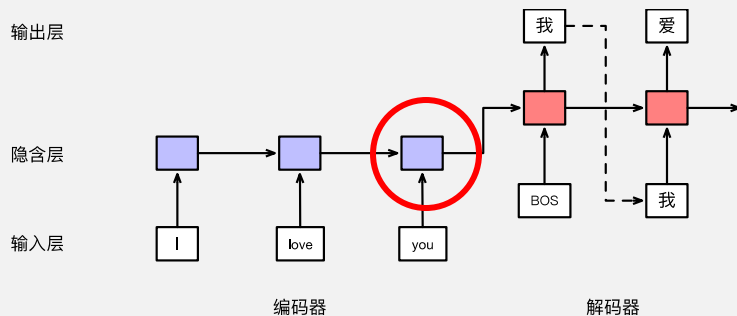


□ 堆叠循环神经网络



□ 序列到序列模型

- 也称“编码器—解码器”模型
- 机器翻译等多种应用



You can't cram the meaning of a whole sentence into a single vector!

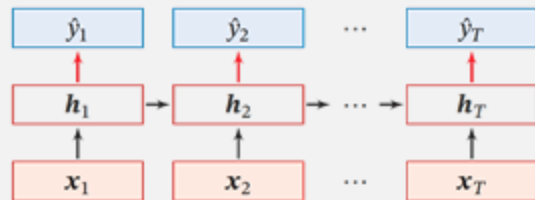


Ray Mooney

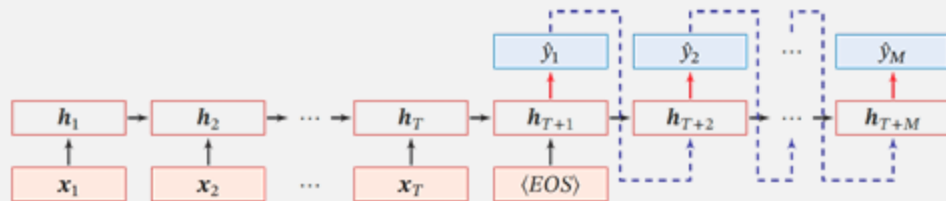


循环神经网络的应用

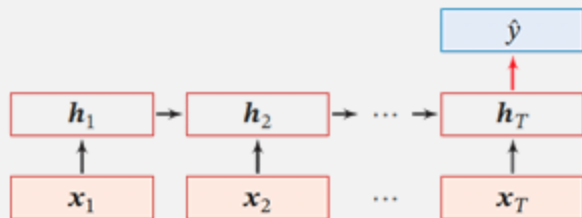
□ 同步的序列到序列模式



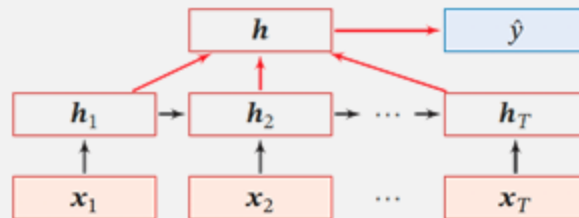
□ 异步的序列到序列模式



□ 序列到类别模式



(a) 正常模式



(b) 按时间进行平均采样模式



大纲

- 多层感知机
- 卷积神经网络
- 循环神经网络
- Transformer
- 神经网络的训练



认知神经学中的注意力

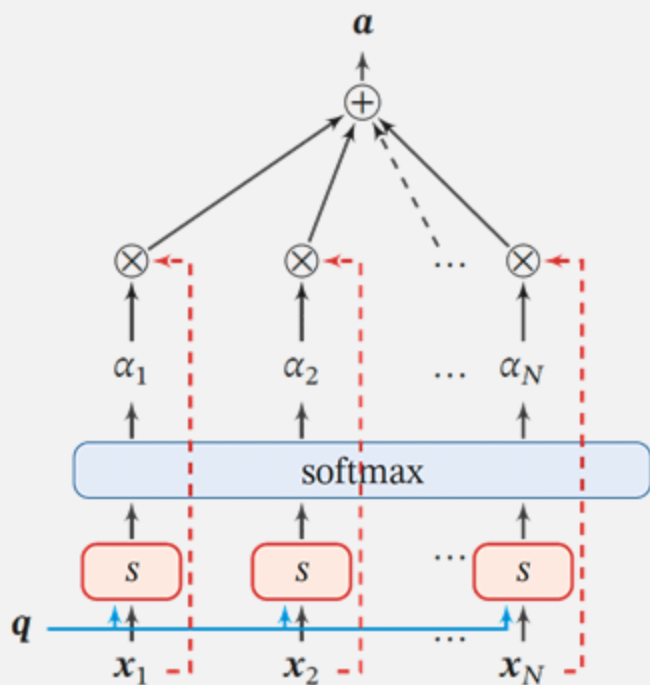
□ 注意力一般分两种：

1. 自上而下的有意识的注意力，称为**聚焦式注意力**（Focus Attention）。
有预定目的、依赖任务的，主动有意识地聚焦于某一对象的注意力。
2. 自下而上的无意识的注意力，称为**显著性注意力**（Saliency-Based Attention）。
由外界刺激驱动的注意，不需要主动干预，也和任务无关。

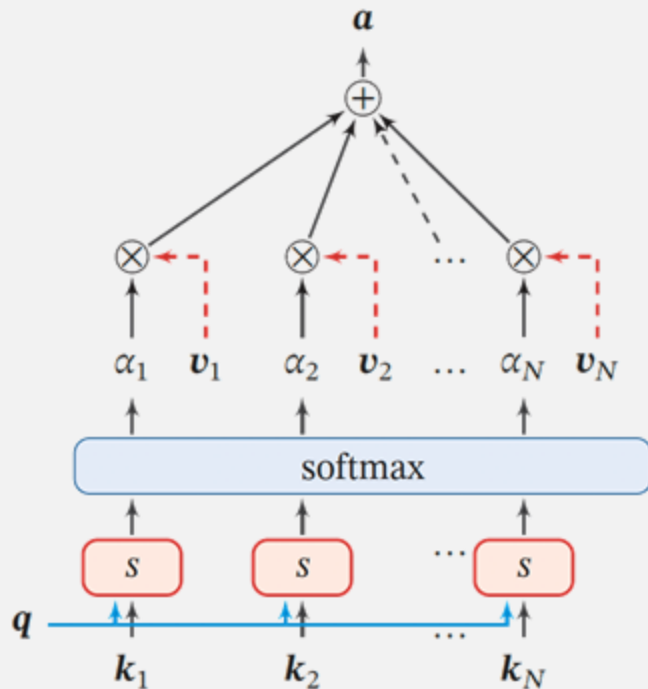
一个和注意力有关的例子是**鸡尾酒会效应**。当一个人在吵闹的鸡尾酒会上和朋友聊天时，尽管周围噪音干扰很多，他还是可以听到朋友的谈话内容，而忽略其他人的声音（**聚焦式注意力**）。同时，如果背景声中有重要的词（比如他的名字），他会马上注意到（**显著性注意力**）。



注意力机制 (Attention Mechanism)



(a) 普通模式

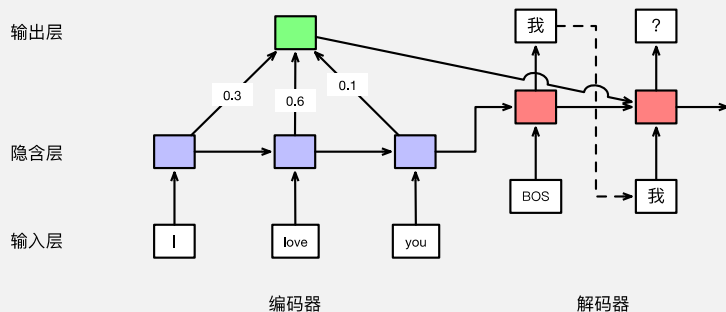


(b) 键值对模式



注意力机制 (Attention Mechanism)

- 当前状态除和前一个状态及输入相关外，还应关注原序列的状态

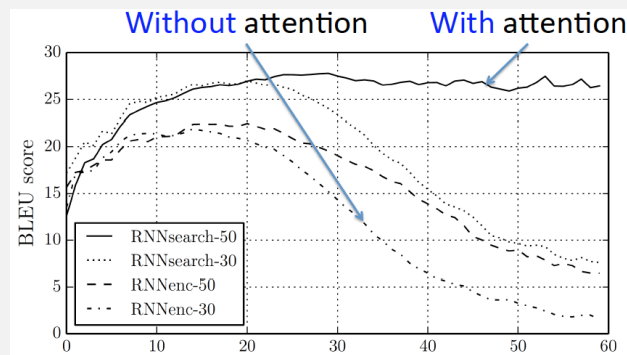
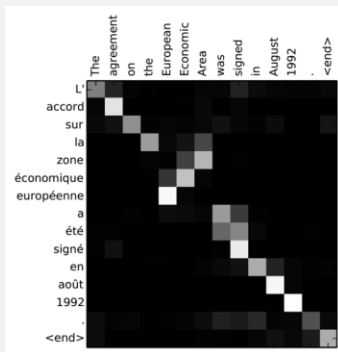


- 打分公式

$$\hat{\alpha}_s = \text{attn}(\mathbf{h}_s, \mathbf{h}_{t-1})$$

$$\alpha_s = \text{Softmax}(\hat{\alpha})_s$$

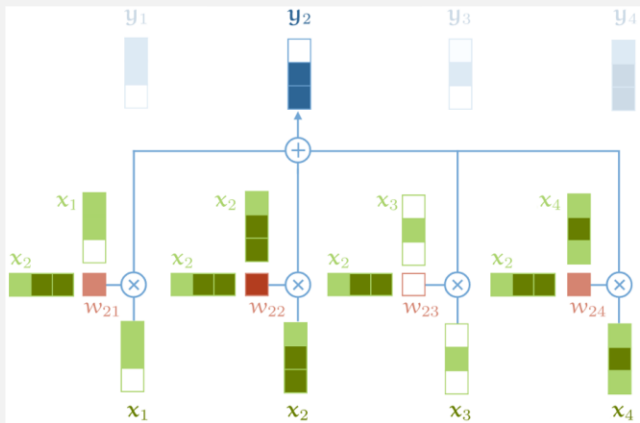
$$\text{attn}(\mathbf{q}, \mathbf{k}) = \begin{cases} \mathbf{w}^\top \tanh(\mathbf{W}[\mathbf{q}; \mathbf{k}]) \\ \mathbf{q}^\top \mathbf{W} \mathbf{k} \\ \mathbf{q}^\top \mathbf{k} \\ \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d}} \end{cases}$$





自注意力模型

- 通过某一输入与周围输入的相关性（注意力）来更新该输入
 - “观其伴、知其义”
- 输入：n个向量构成的序列 $x_1, x_2, \dots, x_n$
- 输出：每个向量对应的新向量 $y_1, y_2, \dots, y_n$
- $y_i = \sum_{j=1}^n \alpha_{ij} x_j$, α_{ij} 为 x_i 与 x_j 之间的注意力值



```
1 import torch
2 import torch.nn.functional as F
3
4 x = torch.rand(3, 4, 5)
5
6 # torch.bmm is a batched matrix multiplication. It
7 # applies matrix multiplication over batches of
8 # matrices.
9
10 raw_weights = torch.bmm(x, x.transpose(1, 2))
11
12 weights = F.softmax(raw_weights, dim=2)
13
14 y = torch.bmm(weights, x)
15
16 print(y)
```



自注意力

Cornell University

We gratefully acknowledge support from the Simons Foundation and member institutions.

arXiv > cs > arXiv:1706.03762

Search... All Fields Search

Help | Advanced Search

Computer Science > Computation and Language

[Submitted on 12 Jun 2017 (v1), last revised 6 Dec 2017 (this version, v5)]

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Download:

- PDF
- Other formats

Current browse context: cs.CL < prev | next > new | recent | 1706 Change to browse by: cs cs.LG

References & Citations

- NASA ADS
- Google Scholar
- Semantic Scholar

84 blog links (what is this?)

DBLP - CS Bibliography

listing | bibtext

Ashish Vaswani
Noam Shazeer
Niki Parmar
Jakob Uszkoreit

Attention Visualizations

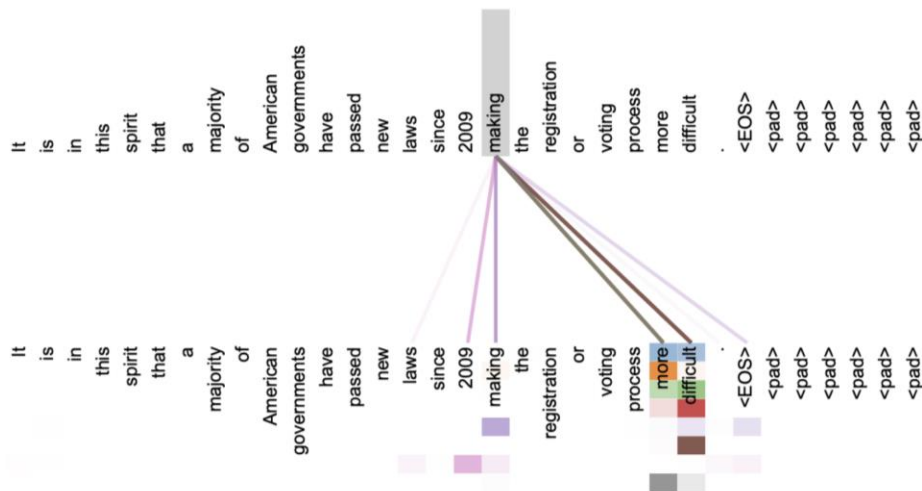


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.



Transformer

- Vaswani et al., Attention Is All You Need, NIPS 2017
- Transformer的翻译
 - 变压器?
 - 变形金刚?
- 自注意力模型还需要解决的几个问题
 - 没有考虑输入的**位置**信息
 - 输入向量 x_i 同时承担三种**角色**, 不易学习
 - 计算注意力权重时的两个向量以及被加权的向量
 - 只考虑了两个输入向量之间的关系, 无法建模多个向量之间的**更复杂关系**
 - 自注意力计算结果互斥, 无法**同时关注**多个输入





融入位置信息

□ 位置嵌入 (Position Embeddings)

- 类似词嵌入，每个绝对位置赋予一个连续、低维、稠密的向量表示
- 向量参数参与模型学习

□ 位置编码 (Position Encodings)

- 直接将一个整数（位置索引值）映射为一个向量

$$\text{PosEnc}(p, i) = \begin{cases} \sin\left(\frac{p}{10000^{\frac{i}{d}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{p}{10000^{\frac{i-1}{d}}}\right) & \text{else} \end{cases}$$

□ 输入向量 = 词向量 + 位置嵌入/编码



赋予输入向量角色信息

□ 输入向量的三种角色

- 查询 (Query)
- 键 (Key)
- 值 (Value)

□ 分别对输入向量进行线性映射

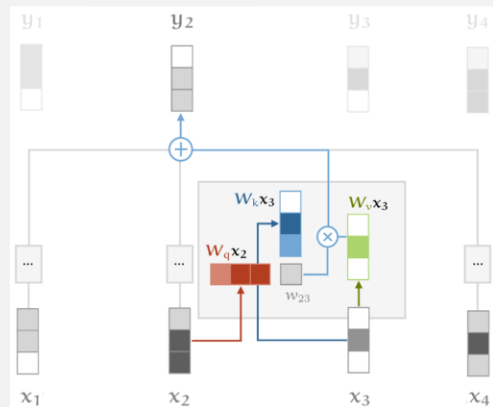
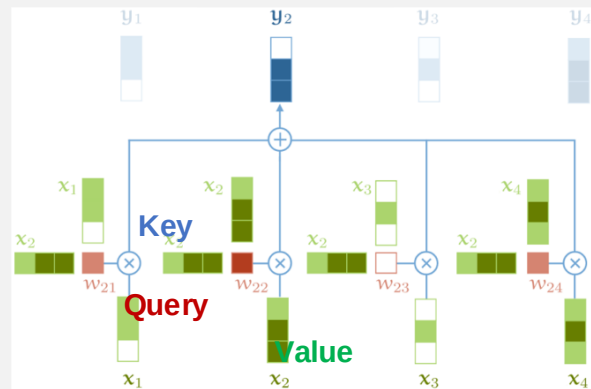
- $q_i = W^q x_i$
- $k_i = W^k x_i$
- $v_i = W^v x_i$

□ 新的自注意力模型

$$y_i = \sum_{j=1}^n \alpha_{ij} v_j$$

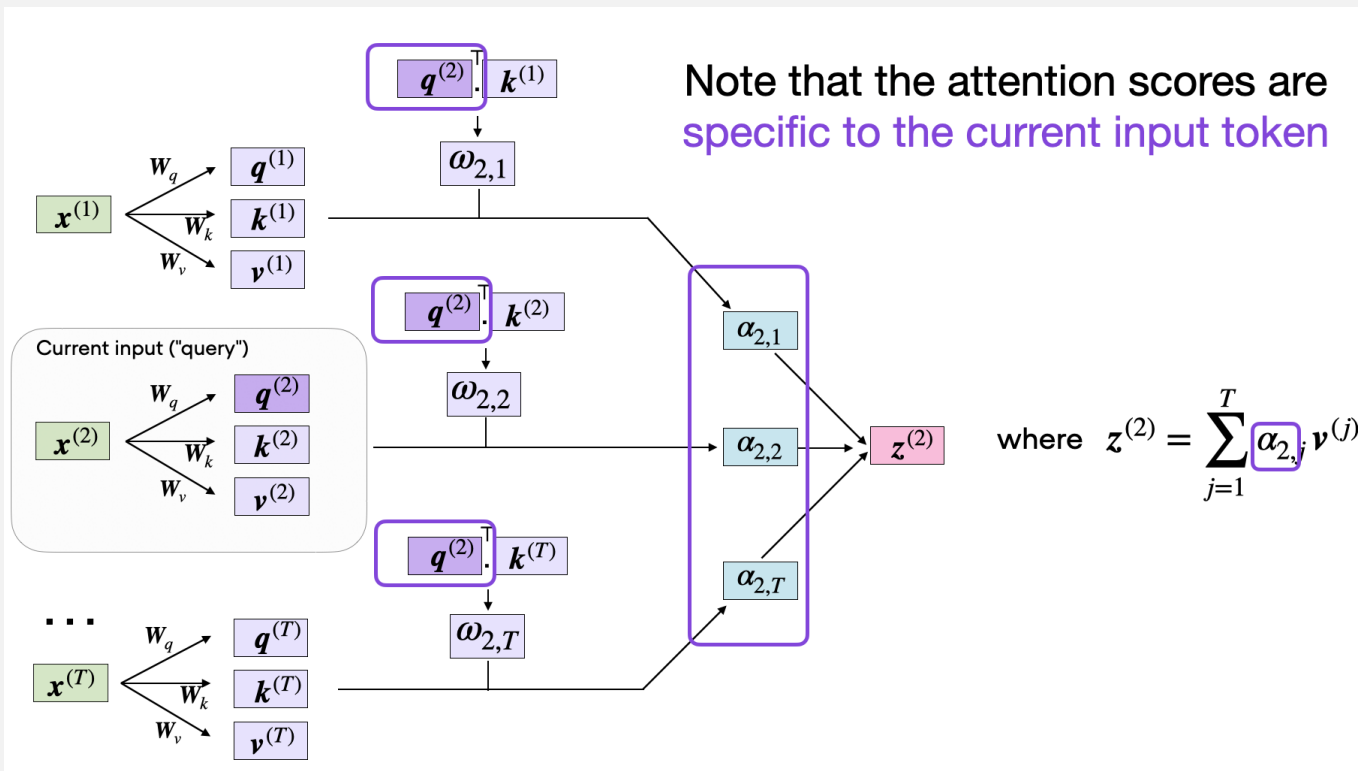
$$\alpha_{ij} = \text{Softmax}(\hat{\alpha}_i)_j$$

$$\hat{\alpha}_{ij} = \text{attn}(q_i, k_j)$$



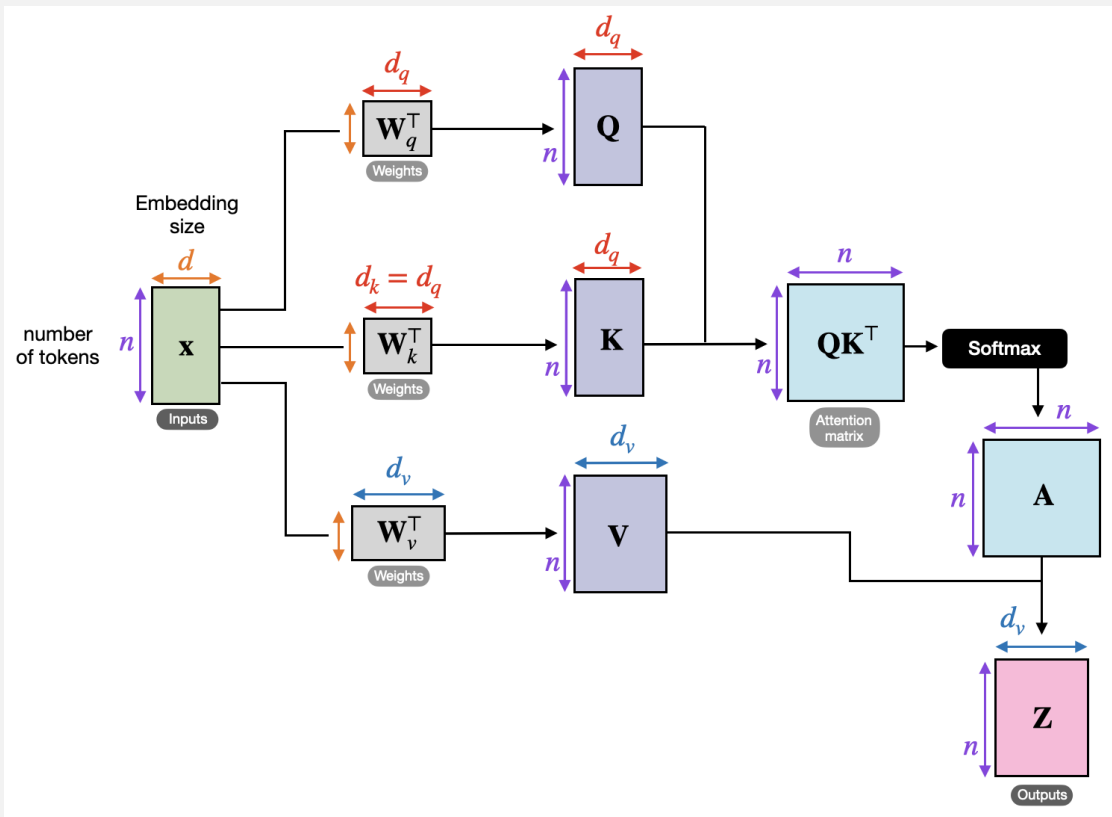
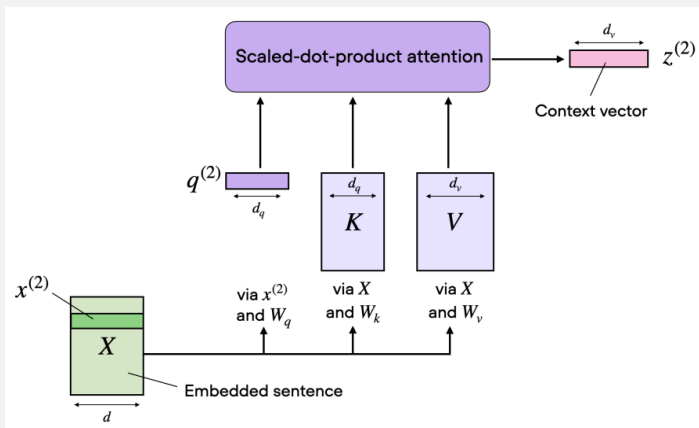


自注意力





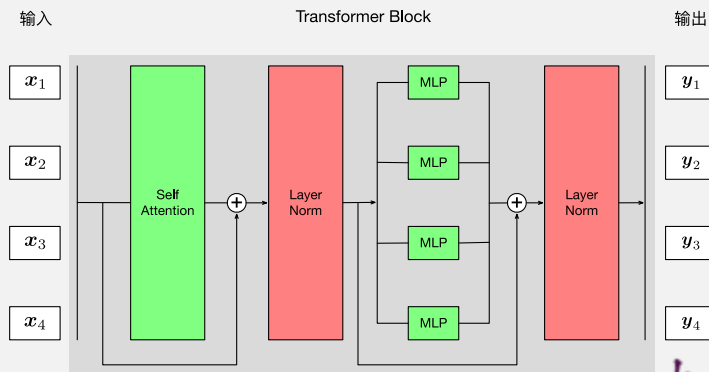
自注意力





多层自注意力

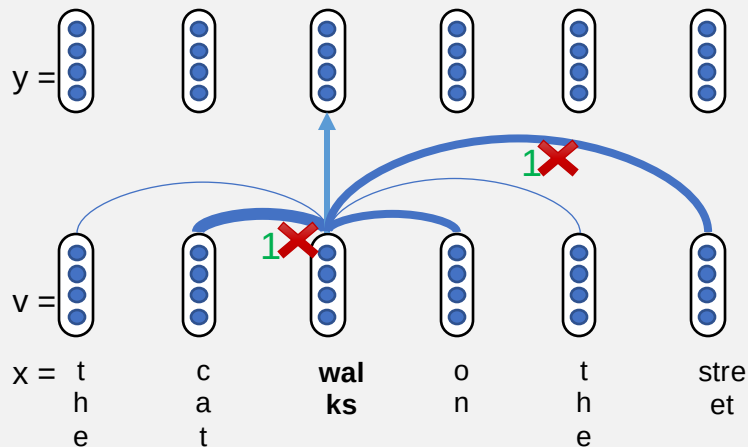
- 原始自注意力模型仅考虑了任意两个向量之间的关系
- 如何建模高阶关系？
 - 直接建模高阶关系导致模型复杂度过高
 - 堆叠多层自注意力模型（消息传播机制）
- 增强模型的表示能力——增加非线性
 - 增加非线性的多层感知器模型（MLP）
- 使模型更容易学习
 - 层归一化（Layer Normalization）
 - 残差连接（Residual Connections）





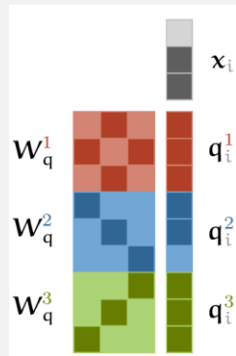
多头自注意力

□ 由于Softmax函数的性质，无法使得多个自注意力分数趋近于1



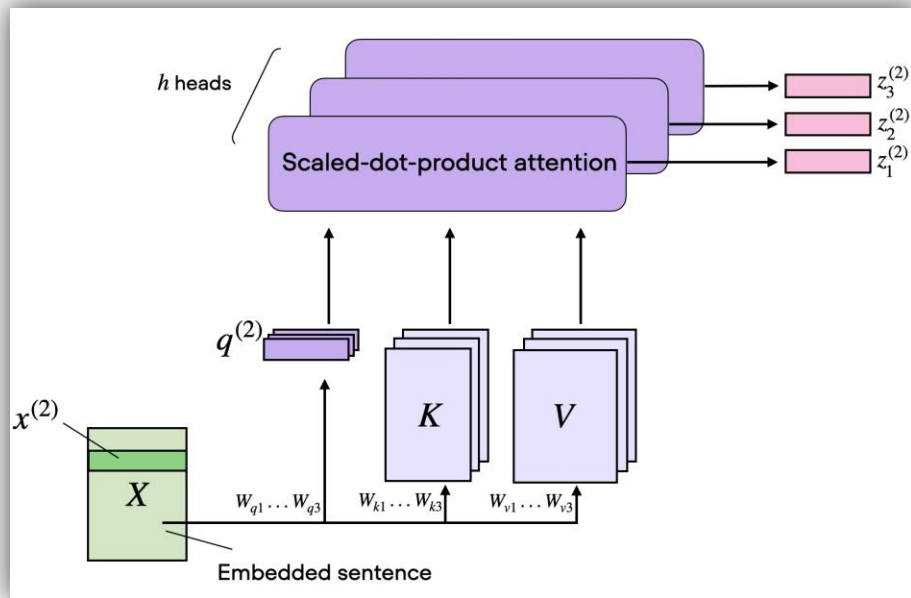
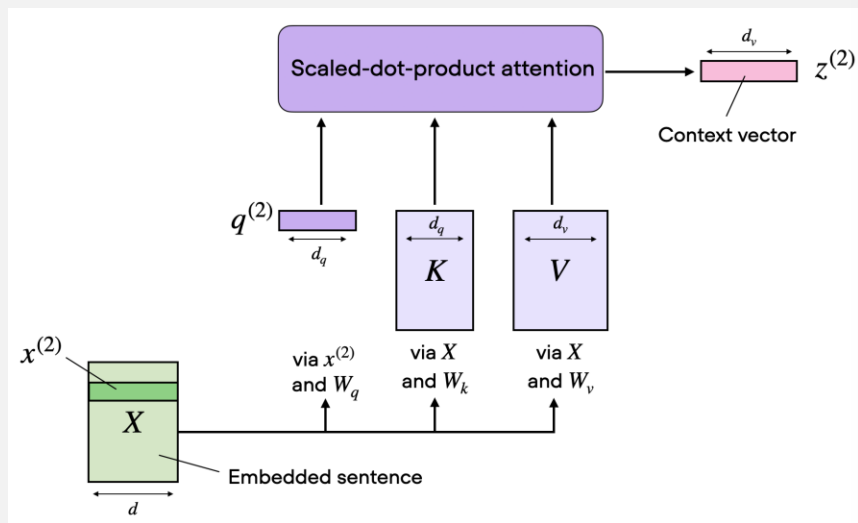
□ 使用多组自注意力模型产生多组不同的注意力结果

- 设置多组输入映射矩阵
- 类似使用多个卷积核提取不同的特征





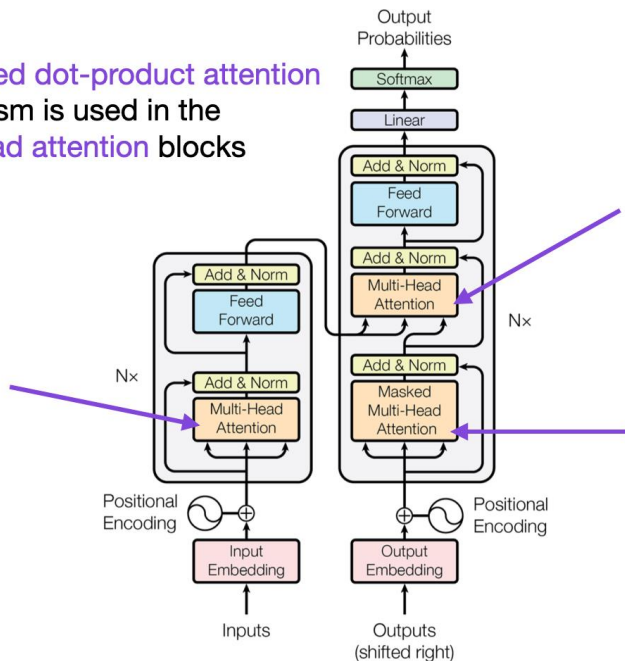
多头自注意力



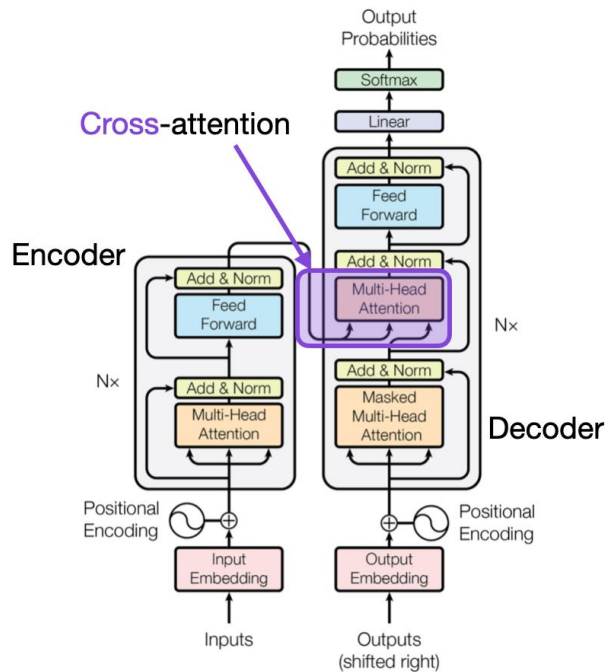


自注意力 & 交叉注意力

The **scaled dot-product attention** mechanism is used in the **multi-head attention** blocks



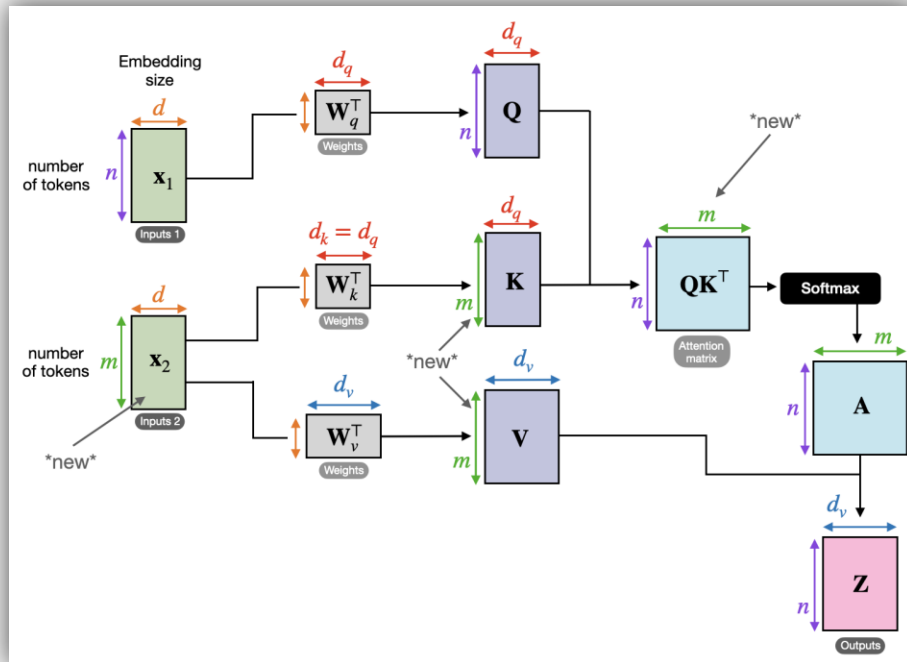
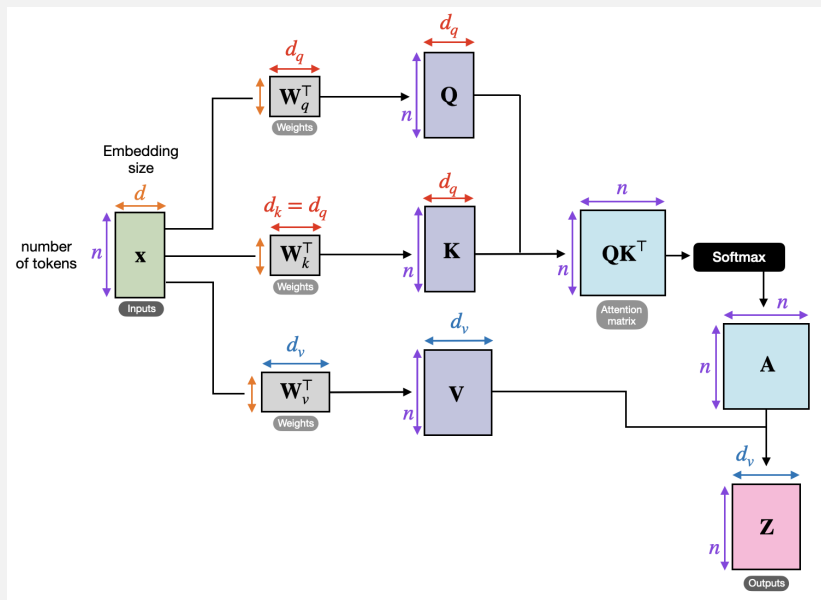
Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)



Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)



交叉注意力 Cross-Attention





Transformer的优缺点

优点

- 直接建模**更长**距离的依赖关系
- **更快**的训练速度（与RNN相比）

缺点

- 参数量**过大**导致模型不容易训练
- 需要基于大规模数据**预训练**





基于Transformer的模型

	Layers	Width	Heads	Params
Transformer-Base	12	512	8	65M
GPT-1	12	512	12	117M
BERT-Large	24	1024	16	340M
XLNet-Large	24	1024	16	340M
RoBERTa	24	1024	16	355M
GPT-2	48	1600	*48	1.5B
Megatron-LM	72	3072	32	8.3B
Turing-NLG	78	4256	28	17B
LaMDA	64	8192	128	137B
GPT-3	96	12,228	96	175B
Gopher	80	16,384	128	280B
PaLM	118	18,432	48	540B





大纲

- 多层感知机
- 卷积神经网络
- 循环神经网络
- Transformer
- 神经网络的训练



神经网络模型的训练

- 寻找一组**优化**的模型参数
 - 又叫做模型**训练**或**学习**
- 损失函数 (Loss Function)
 - 评估参数好坏的**准则**
 - 为什么不直接使用**准确率**等指标进行评估?
- 两种常用的损失函数

- **均方误差** (Mean Squared Error, MSE)

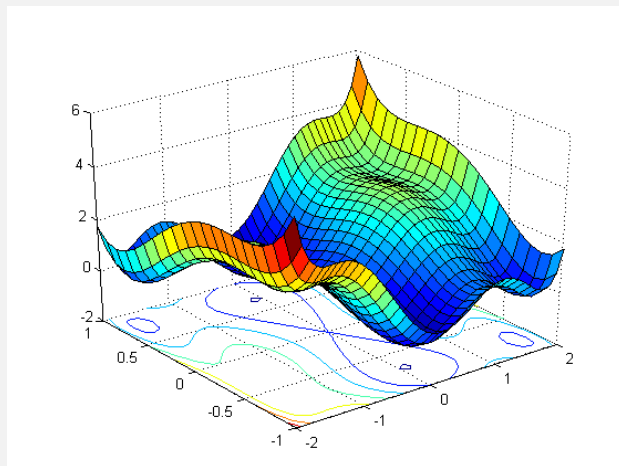
$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$$

- **交叉熵** (Cross-Entropy, CE)

$$\text{CE} = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^c y_j^{(i)} \log \hat{y}_j^{(i)} \quad \Rightarrow \quad \text{CE} = -\frac{1}{m} \sum_{i=1}^m \log \hat{y}_t^{(i)}$$

负对数似然损失

Negative Log Likelihood, NLL





梯度下降

□ 梯度 (Gradient)

- 以向量的形式写出的对多元函数各个参数求得的偏导数
- 是函数值增加最快的方向
- 沿着梯度相反的方向，更容易找到函数的极小值

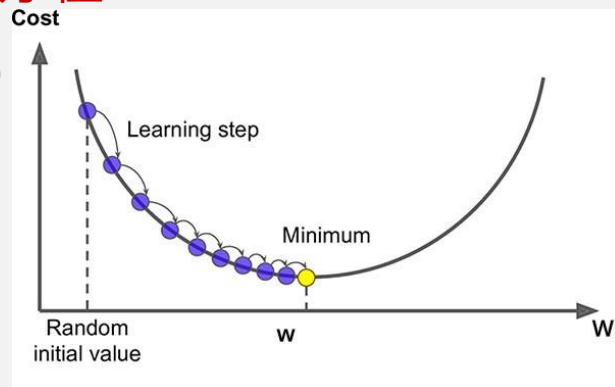
□ 梯度下降算法 (Gradient Descent, GD)

算法 4.1 梯度下降算法

Input: 学习率 α ; 含有 m 个样本的训练数据

Output: 优化参数 θ

1. 设置损失函数为 $L(f(x; \theta), y)$;
2. 初始化参数 θ_0 ;
3. **while** 未达到终止条件 **do**
4. 计算梯度 $g = \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$;
5. $\theta = \theta - \alpha g$;
6. **end**



□ 小批次梯度下降法 (Mini-batch Gradient Descent)

- 每次随机采样小规模的训练数据来估计梯度
- 提高算法的运行速度



PyTorch代码示例

```
# 创建多层感知器模型, 输入层大小为2, 隐含层大小为5, 输出层大小为2 (即有两个类别)
model = MLP(input_dim=2, hidden_dim=5, num_class=2)

criterion = nn.NLLLoss() # 当使用log_softmax输出时, 需要调用负对数似然损失 (Negative Log Likelihood, NLL)
optimizer = optim.SGD(model.parameters(), lr=0.05) # 使用梯度下降参数优化方法, 学习率设置为0.05

for epoch in range(500):
    y_pred = model(x_train) # 调用模型, 预测输出结果
    loss = criterion(y_pred, y_train) # 通过对比预测结果与正确的结果, 计算损失
    optimizer.zero_grad() # 在调用反向传播算法之前, 将优化器的梯度值置为零, 否则每次循环的梯度将进行累加
    loss.backward() # 通过反向传播计算参数的梯度
    optimizer.step() # 在优化器中更新参数, 不同优化器更新的方法不同, 但是调用方式相同
```





小结

- 感知机模型
- Logistic 回归模型
- Softmax 回归模型
- 多层感知机模型
- 卷积神经网络
- 循环神经网络
- 长短时记忆网络 (LSTM)
- 注意力机制
- 自注意力模型
- Transformer
- 神经网络的训练



参考材料

■ CNN

- <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>

■ RNN

- <https://medium.com/@YanAlx/understand-recurrent-neural-network-with-four-figures-d78293a76294>
- <https://medium.com/analytics-vidhya/recurrent-neural-network-and-its-variants-de75f9ee063>

■ Transformer

- <https://builtin.com/artificial-intelligence/transformer-neural-network>
- <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>

